

## Lista de Exercícios – Evolução das Principais Linguagens de Programação

Vanessa Beatriz G. Beltrame. 24034783-2.

### Questão 1

Dizer que a evolução das linguagens não é uma escada de progresso significa que uma linguagem nova não faz com que as antigas deixem de ser usadas. Muitas linguagens continuam sendo usadas dependendo da situação. Por exemplo, COBOL ainda é usada em sistemas antigos de bancos e Fortran ainda aparece bastante em cálculos científicos. Então, a evolução parece mais uma árvore, com várias linguagens seguindo caminhos diferentes.

Dois fatores que ajudam nisso são:

O hardware da época: algumas linguagens foram criadas pensando nas limitações dos computadores daquele período. Algumas dessas escolhas ainda são úteis em determinadas situações.

A área em que a linguagem é usada: cada linguagem pode ser melhor para um tipo de problema. Por exemplo, COBOL foi feita pensando em sistemas comerciais e Lisp ficou conhecida por trabalhar com inteligência artificial e manipulação de símbolos.

### Questão 2

A Plankalkül é importante porque foi uma linguagem pensada por Konrad Zuse entre 1936 e 1945 e já tinha algumas ideias que só ficaram comuns em linguagens muitos anos depois. Mesmo não tendo sido implementada naquela época, ela mostrou que já existiam ideias de programação mais avançadas.

Três recursos que ela já tinha eram:

Estruturas de dados, como registros e arrays.

Atribuição de valores e formas de repetição.

Uma forma de trabalhar com invariantes e afirmações sobre os dados.

As estruturas de dados são importantes porque mostram que já existia a ideia de trabalhar com informações mais complexas, e não somente com valores separados.

### Questão 3

Os três sistemas tentavam resolver um problema parecido, que era facilitar a programação, já que programar diretamente em linguagem de máquina era muito difícil e demorado.

Short Code: permitia escrever cálculos de uma forma mais parecida com a matemática, mas era interpretado, então acabava sendo mais lento.

Speedcoding: também era um interpretador e foi criado para facilitar principalmente cálculos com números e ponto flutuante.

A-0, A-1 e A-2: foram desenvolvidos por Grace Hopper e já tinham uma ideia mais próxima de um compilador, pois transformavam instruções simbólicas em código de máquina.

Não é muito correto chamar todos eles de compiladores modernos porque eles ainda eram bem limitados. Alguns funcionavam por interpretação e não tinham os recursos e otimizações dos compiladores atuais. A ideia de compilação ficou mais próxima do que conhecemos hoje com linguagens como Fortran.

## Questão 7

COBOL foi criada para trabalhar principalmente com sistemas comerciais, como folha de pagamento, estoque e faturamento. Por isso, ela foi pensada para ser mais fácil de ler, até para pessoas que não eram programadores.

Legibilidade: os comandos usam palavras parecidas com o inglês comum, como ADD AMOUNT TO TOTAL, deixando o código mais fácil de entender.

Registros: COBOL precisava trabalhar com vários tipos de informações juntas, como nomes, endereços, datas e valores. Por isso, os registros eram importantes.

FLOW-MATIC: COBOL teve bastante influência da FLOW-MATIC, que também usava palavras em inglês nos comandos e era voltada para processamento de dados comerciais.

Então, o objetivo da COBOL influenciou bastante a forma como ela foi criada.

## Questão 8

BASIC e PL/I queriam aumentar o uso da programação, mas fizeram isso de maneiras diferentes.

A BASIC foi criada para facilitar o acesso à programação, principalmente para estudantes e pessoas que não eram especialistas em computação. O ponto positivo era a simplicidade, mas isso também fazia com que ela tivesse menos recursos para programas muito grandes.

Já a PL/I tentou juntar várias coisas em uma linguagem só. Ela queria atender tanto à computação científica, como o Fortran, quanto ao processamento comercial, como o

COBOL. O problema é que, por ter muitos recursos, acabou ficando uma linguagem mais complexa.

Resumindo: BASIC priorizou a facilidade de aprender, enquanto PL/I priorizou ter mais recursos.

## Questão 12

Um exemplo de uma pequena base em Prolog seria:

Fato 1: Maria é mãe de João.

Fato 2: João é pai de Pedro.

Regra: X é avó de Z se X é mãe de Y e Y é pai de Z.

Consulta: Quem é avó de Pedro?

Nesse caso, o Prolog consegue descobrir que Maria é avó de Pedro usando os fatos e a regra.

Isso é programação lógica porque o programador não precisa explicar passo a passo como encontrar a resposta. Ele coloca os fatos e as regras, e o Prolog faz a busca para descobrir a resposta.

Se fosse apenas um armazenamento de dados, Maria ser avó de Pedro precisaria estar registrada diretamente. No Prolog, essa informação pode ser descoberta através da regra.

## Questão 14

As três linguagens usam objetos de maneiras diferentes.

Smalltalk colocou os objetos como a parte principal da linguagem. Praticamente tudo nela é tratado como objeto.

C++ adicionou recursos de orientação a objetos ao C. A ideia era continuar tendo compatibilidade com códigos C que já existiam. Isso foi bom para quem já programava em C, mas também deixou a linguagem com mais de um estilo de programação.

Java também usa orientação a objetos, mas tentou deixar algumas coisas mais simples que no C++. Além disso, ela usa a JVM. O programa é transformado em bytecode e pode ser executado em diferentes sistemas que tenham uma JVM.

Então, de forma simples: Smalltalk colocou os objetos no centro, C++ adicionou objetos ao C e Java usou a máquina virtual para facilitar a portabilidade.

## Questão 15

Java não foi criada originalmente pensando na Web. Ela começou como um projeto chamado Oak e tinha como objetivo ser usada em dispositivos eletrônicos, como aparelhos e controles.

Depois, a Web começou a crescer bastante e perceberam que algumas características da Java poderiam ser úteis nela, principalmente a portabilidade e a possibilidade de executar o mesmo programa em sistemas diferentes.

Com isso, Java passou a ser usada na Web, principalmente com os applets, e acabou ficando muito mais conhecida.

Isso mostra que uma linguagem pode acabar sendo usada em uma área diferente daquela para a qual foi criada. Quando aparece uma nova necessidade, uma linguagem que já existe pode acabar sendo uma boa solução.

## Questão 16

As linguagens de scripting não são todas iguais. Cada uma surgiu para resolver problemas diferentes.

| Linguagem Domínio inicial |                                                  | Estruturas de dados                   | Implementação                                     |
|---------------------------|--------------------------------------------------|---------------------------------------|---------------------------------------------------|
| Perl                      | Textos e administração de sistemas Unix          | Arrays, hashes e expressões regulares | Interpretada, com uma representação intermediária |
| JavaScript                | Interatividade em páginas Web                    | Objetos e arrays                      | Interpretada e também pode usar JIT               |
| PHP                       | Páginas Web dinâmicas                            | Arrays associativos                   | Interpretada no servidor                          |
| Python                    | Uso geral, ensino e prototipagem                 | Listas, dicionários e tuplas          | Interpretada, usando bytecode no CPython          |
| Ruby                      | Programação orientada a objetos e produtividade  | Tudo é tratado como objeto            | Interpretada                                      |
| Lua                       | Ser usada dentro de outros programas, como jogos | Tabelas                               | Interpretada por uma máquina virtual leve         |

Mesmo sendo chamadas de linguagens de scripting, elas têm diferenças. Perl ficou muito ligada a textos e Unix, JavaScript à Web, PHP ao servidor, Python ao uso geral, Ruby à orientação a objetos e Lua à possibilidade de ser colocada dentro de outros programas.

## Questão 18

XSLT: recebe um documento XML como entrada e usa regras para transformar esse documento. A saída pode ser outro XML, HTML ou texto.

JSP: mistura HTML com código Java. O código é executado no servidor e depois é gerada uma página que será enviada para o navegador.

As duas podem ser consideradas híbridas porque misturam marcação com alguma forma de programação. No XSLT existem regras e condições para fazer a transformação, enquanto no JSP podemos colocar código Java junto com o HTML.

Por isso, elas não são apenas linguagens de marcação, mas também possuem recursos para fazer processamento.